

LifeSystem

AI Personal Guidance & Real-Life Progression System

A serious major-project blueprint for building a system that helps people understand their current situation, choose the next useful action, recover when needed, and improve over time.

Core idea	Understand me -> guide me -> help me act -> learn from the result -> adapt -> level up.
Primary domains	Physical, Educational, Mental Wellness, Emotional Wellbeing, Career / Life Direction
Signature engine	Next Best Action + Bottleneck Detection + Adaptive Progression
Positioning	Not an anime clone and not just a chatbot. A personal guidance engine with RPG-style progression.

Blueprint status: v1.0 | Designed for a 3rd-year student building toward a deployable portfolio-quality major project.

1. Executive Product Definition

LifeSystem is an AI-assisted personal guidance platform. A user tells the system about their goals, habits, learning, current state and constraints. The system builds a personal profile, detects the most important bottleneck, selects a next best action, gives a manageable quest or recovery plan, measures the outcome, and updates future recommendations.

The core product decision is not “what tasks can I generate?” It is “given this person, right now, what is the most useful safe next step?”

Product layer	Purpose
Assessment	Understand baseline, goals, constraints and preferences.
Personal state	Represent current domain scores, confidence, energy and workload.
Decision engine	Find bottlenecks, priorities, dependencies and the next best action.
Quest engine	Turn the action into a concrete task with difficulty, duration and reward.
Recovery mode	Reduce pressure and recommend safe wellbeing-support actions when the user reports low energy, stress or emotional difficulty.
Progress engine	Use evidence and feedback to update scores, XP, skills, streaks and recommendations.
AI layer	Explain, personalize and generate constrained language/content; it does not control safety rules.

2. Product Principles & Non-Goals

These rules should remain true even when new features are added.

Principle	What it means
Guidance before gamification	XP is a motivation layer; the real value is better decisions and support.
Personal before generic	The system should adapt to goals, ability, time, energy and history.
Small actionable steps	Recommendations should be realistic and specific enough to start immediately.
Explainable recommendations	The user should be able to see why a recommendation was selected.
Evidence over clicks	Progress should rely on outcomes where possible, not only "Completed" buttons.
Sustainable progress	Recovery is a valid system state; the app should not reward harmful overwork.
Privacy by design	Collect the minimum necessary data, protect it, and give users control.

Non-goals

- Do not diagnose medical or psychiatric conditions.
- Do not claim that a score is a clinical measurement.
- Do not make the LLM the source of truth for safety-critical decisions.
- Do not build every advanced feature before proving the core loop.

3. Target Users & Jobs-to-be-Done

Primary users are students and early-career adults who have goals but do not always have consistent access to a mentor, coach or structured planning system.

User situation	Need
Confused about career	Understand options, choose a direction, get a roadmap and next step.
Exam preparation	Know what to study first, track learning evidence and adjust workload.
Inconsistent routine	Turn vague goals into manageable quests and recover after missed days.
Feeling tired/stressed	Switch from performance mode to supportive recovery mode.
Making progress	See concrete evidence of growth and what remains to improve.

Core user journey

Onboard -> assess -> define goal -> receive status -> check in -> get next best action -> complete or modify quest -> record result -> receive feedback -> level up -> repeat.

4. The Five Personal Development Domains

Physical - Activity, strength/endurance habits, mobility, sleep routine, recovery and other user-reported wellness behaviors. Use as lifestyle/fitness guidance, not diagnosis.

Educational - Subjects, skills, learning goals, study consistency, quizzes, coding problems, project milestones and demonstrated knowledge.

Mental Wellness - Focus, workload, perceived stress, recovery habits, attention and self-management. Frame outputs as wellbeing support, not diagnosis.

Emotional Wellbeing - Mood check-ins, journaling, social connection, emotional awareness and coping activities. Handle sensitive signals conservatively.

Career / Life Direction - Goals, role interests, skill gaps, roadmaps, milestones, portfolio projects and decision support.

Domain scores should be interpreted as gamified progress indicators, not clinical or scientific measurements unless a validated instrument is deliberately incorporated and its limits are clearly stated.

5. Core Product Loop

This is the heart of the product and should be fully working before advanced features are added.

**USER DATA -> PERSONAL STATE -> BOTTLENECK DETECTION ->
NEXT BEST ACTION -> QUEST / RECOVERY -> RESULT ->
FEEDBACK -> UPDATED STATE**

Step	System responsibility	Output
1. Check-in	Collect current energy, stress, mood, available time and constraints.	Current-state snapshot
2. Analyze	Combine baseline, goals, recent behavior and performance.	Candidate priorities
3. Prioritize	Choose the action with the highest expected value and acceptable effort/risk.	One primary next action
4. Generate	Convert action into a concrete quest/recovery activity.	Task + duration + difficulty + reward
5. Explain	Tell the user why it was selected.	Reasoning summary
6. Measure	Capture completion, quality, difficulty and user feedback.	Evidence + feedback
7. Adapt	Update future difficulty and priority.	Next-state model

6. Assessment System

Onboarding should establish enough context to personalize without becoming an exhausting survey. Use progressive disclosure: essential questions first, deeper questions later.

Category	Example inputs
Identity & basics	Age range, preferred language, role/student status, timezone.
Goals	Short-term goal, long-term goal, target date, motivation.
Physical	Activity level, exercise experience, sleep routine, current limitations.
Education	Current subjects, skill levels, upcoming exams, learning preference.
Wellbeing	Energy, perceived stress, recovery habits, mood check-in, support preferences.
Career	Target field/role, current skills, desired timeline, portfolio status.
Constraints	Available hours, schedule, travel, device access, budget, preferred activities.

Baseline status output

Create a status card containing Master Level, XP, domain indicators, active goals, current state, bottleneck, and today's recommended action. Avoid false precision: a progress indicator such as 62/100 should be described as a system score, not a clinical assessment.

7. Next Best Action Engine

This is the product's main differentiator. It combines rule-based logic with AI assistance rather than asking an LLM to invent decisions from scratch.

Conceptual scoring model

$$\text{Priority} = \text{Need} \times \text{GoalImpact} \times \text{Readiness} \times \text{EvidenceGap} \times \text{Timeliness} - \text{EffortPenalty} - \text{OverloadPenalty}$$

Where each factor is normalized to a consistent range. The exact weights should be tested and revised using user feedback.

Signal	Meaning
Need	How much the area currently needs attention.
Goal impact	How directly the action contributes to a declared goal.
Readiness	Whether the user has the energy/time/prerequisites to act now.
Evidence gap	Whether more evidence is needed before making a strong recommendation.
Timeliness	Deadlines, upcoming exams, interviews or other time-sensitive factors.
Effort penalty	Reduce priority for actions that are too large for current constraints.
Overload penalty	Reduce priority if the user has excessive recent workload or repeated failures.

8. Bottleneck Detection

The system should not simply pick the lowest score. It should look for dependencies and cross-domain effects.

Example

Recovery decline -> low energy -> shorter study sessions -> missed DSA practice -> slower career preparation.

The engine can therefore identify **recovery** as the immediate bottleneck even when education is the visible goal.

Initial implementation

- Start with explicit dependency rules written by you.
- Store bottleneck decisions and the evidence used to make them.
- Later experiment with statistical/ML models once enough user history exists.

This makes the project academically defensible: you can explain the decision path instead of saying “the AI decided.”

9. Quest & Progression Engine

Component	Design
Quest type	Main quest, side quest, recovery quest, learning quest, milestone quest.
Difficulty	Beginner/intermediate/advanced or continuous 0-1 scale internally.
Duration	Estimated minutes; should match user availability.
Reward	XP plus domain-specific progress/skill unlocks.
Evidence	Quiz score, solved problems, logged session, reflection, habit consistency, etc.
Failure	Never shame; reduce or modify difficulty and capture reason.
Leveling	Level thresholds, skill unlocks, ranks and progress history.

Evidence-based XP

Example: a study session can earn base XP for completion, bonus XP for demonstrated understanding, and a retention bonus after a delayed check. This avoids rewarding only time spent.

Adaptive difficulty loop

High success + low overload -> increase challenge. Low success -> inspect the reason. Repeated overload -> shift to recovery or reduce scope. Consistent success -> unlock harder milestones.

10. Recovery & Support Mode

When a user reports feeling tired, stressed, anxious, overwhelmed or emotionally low, the system should not automatically push harder. It should shift into a safer support mode.

Example flow

User says “I am exhausted today.” -> lightweight check-in -> estimate current capacity -> pause/resize demanding quests -> offer low-risk recovery options -> ask whether the suggestion helped -> adjust tomorrow.

Reported state	Possible support options
Low energy	Short walk, hydration, rest break, lighter task, consistent bedtime routine.
High stress	Breathing/relaxation exercise, short break, simplify the day, talk to a trusted person.
Overwhelmed by study	Shrink the task, choose one priority, use a short focused session, postpone non-critical work.
Lonely / disconnected	Reach out to a trusted person, social activity, supportive reflection.
Needs distraction/recreation	Suggest a short calming video, music, light entertainment or other user-preferred activity without turning avoidance into a long task.

Safety boundary: the system can support wellbeing and encourage professional help when appropriate, but it must not diagnose anxiety, depression, burnout or other conditions.

11. AI Architecture

Use AI where language generation, summarization or flexible personalization is useful. Keep deterministic logic around scoring, permissions and safety.

Recommended separation

Frontend -> API -> Decision Engine -> Safety/Policy Gate -> LLM -> Validation -> Response

Layer	Responsibility
Decision engine	Computes scores, bottlenecks, candidate actions and constraints.
Safety gate	Blocks unsafe instructions, handles escalation rules and sensitive-state logic.
LLM	Turns structured candidates into natural language, explanations, reflections or content suggestions.
Validator	Checks that generated output matches the allowed action schema and domain limits.
Feedback store	Records usefulness, acceptance, completion and user corrections.

Do not send unnecessary personal information to the model. Build prompt templates around structured fields and keep secrets/API keys on the server.

12. Content Recommendation System

For relaxation or learning content, recommendation should be based on the user's current state, goal, time and preferences.

Input	Example
State	Stressed, low energy, short break available.
Preference	Comedy, instrumental music, short videos.
Constraint	20 minutes available, needs to study afterward.
Outcome target	Feel calmer without losing the rest of the evening.

Recommendation output should include why the content was chosen and a time limit when appropriate. Avoid presenting a recommendation as guaranteed to “relieve anxiety”; say “may help you relax” or similar.

For the student project, begin with a small curated catalog or metadata-based recommendation layer. Add external integrations only after the core system works.

13. Data Model

A relational model is a good fit for the first production version.

Table	Purpose
users	Identity, preferences, account settings.
profiles	Stable profile and baseline information.
goals	User goals, targets, deadlines and status.
domain_scores	Time-stamped progress indicators by domain.
checkins	Current energy, stress, mood, capacity and context.
quests	Generated tasks, difficulty, reward and status.
quest_results	Completion evidence, quality, duration and feedback.
skills	Skill tree nodes and unlock states.
roadmaps	Goal-to-skill roadmaps and milestones.
recommendations	Selected next actions and explanations.
feedback	User acceptance, usefulness, correction and comments.
content	Curated movies/videos/music/articles/exercises with metadata.
safety_events	Non-diagnostic safety flags, interventions and audit trail.

14. Technical Architecture

Recommended stack for your current background and job-oriented learning path:

Layer	Recommended technology	Why
Frontend	React + JavaScript/TypeScript	Modern job-relevant UI skills.
Backend	Python + FastAPI	Fits your Python background and supports clean APIs.
Database	PostgreSQL + SQLAlchemy + Alembic	Strong relational foundation and migrations.
AI	LLM API via backend	Flexible natural-language generation and explanation.
Analytics	Python + Pandas + Matplotlib	Offline analysis and model evaluation.
ML later	scikit-learn	Start with interpretable models before deep learning.
Auth	JWT/session-based auth	Secure access control.
Deployment	Docker + a cloud host	Reproducible deployment and portfolio readiness.
Testing	Pytest + API tests + frontend tests	Regression protection as the system grows.

High-level architecture

React UI -> FastAPI -> Auth / Domain services / Decision Engine / AI Gateway -> PostgreSQL.
Optional background jobs handle scheduled summaries, reminders and analytics.

15. API Surface (Initial)

Method	Endpoint	Purpose
POST	/auth/register	Create account.
POST	/auth/login	Authenticate user.
GET	/me	Fetch current profile.
POST	/assessment	Submit/update assessment.
GET	/status	Return current system state.
POST	/checkins	Create daily/current-state check-in.
POST	/next-action	Return the recommended next action.
GET	/quests/today	Fetch today's quests.
POST	/quests/{id}/result	Record quest outcome.
GET	/progress	Return progress analytics.
GET	/roadmaps	Fetch current goal roadmaps.
POST	/feedback	Record recommendation/quest feedback.

Use a versioned API path later (for example /api/v1/) once the interface stabilizes.

16. Screen-by-Screen UX Blueprint

Screen	Purpose	Must show
Welcome	Explain the product and boundaries.	Value proposition + privacy/safety basics.
Assessment	Build baseline.	Progress indicator + save/resume.
Status	User's home base.	Level, XP, domains, current state, bottleneck, next action.
Today	Make action obvious.	Primary quest, side quests, recovery state, why it was chosen.
Quest detail	Help user act.	Steps, duration, difficulty, evidence, reward.
Check-in	Capture current condition.	Energy, stress, mood, capacity, optional note.
Progress	Show change over time.	Domain trends, evidence, streaks, milestones.
Roadmap	Connect daily action to long-term goals.	Goal -> skills -> milestones -> quests.
Recovery	Support without pressure.	Calming/recovery options and escalation guidance when needed.
Settings	Control data and preferences.	Privacy, delete/export, content preferences, notification settings.

17. Evaluation & Success Metrics

A serious project needs evidence that the system is useful. Use a test cohort, synthetic data, or controlled self-testing initially, then expand carefully.

Metric	What it tells you
Quest acceptance rate	Whether recommendations feel relevant enough to start.
Quest completion rate	Whether difficulty and effort are realistic.
Usefulness score	Whether users report that the recommendation helped.
Reason acceptance	Whether users agree with "why this was selected."
Adaptation quality	Whether difficulty changes improve success without overload.
Retention	Whether users return over weeks.
Safety test pass rate	Whether risky or disallowed recommendation scenarios are handled correctly.
System latency	Whether recommendations are fast enough to feel interactive.

Research question you can use in the report

Does a context-aware next-best-action system improve task completion and perceived usefulness compared with a generic fixed checklist?

18. Security, Privacy & Safety Requirements

- Hash passwords with a modern password hashing algorithm; never store plaintext passwords.
- Keep API keys and secrets server-side; use environment variables/secret storage.
- Use authentication + authorization on every protected endpoint.
- Encrypt data in transit and protect sensitive data at rest where appropriate.
- Provide data export and deletion controls.
- Log safety decisions and critical events without storing unnecessary sensitive text.
- Use input validation and rate limiting to reduce abuse.
- Separate wellness support from diagnosis/treatment claims.
- Define escalation paths for high-risk user statements; ordinary gamification should never override safety.

Privacy principle

Collect only what is necessary to provide the feature, explain why the data is used, and allow the user to control retention where practical.

19. Product Build Phases

Phase	Build outcome	Exit criteria
0. Foundations	Repo, backend shell, database, basic frontend.	Local stack runs reproducibly.
1. Assessment	Onboarding + profile + baseline status.	A new user can complete onboarding and see a status card.
2. Progression	XP, levels, domain scores, quests.	A user can complete a quest and see progress update.
3. Intelligence	Bottleneck + next-best-action engine + explanations.	System can justify and adapt recommendations.
4. Recovery	Check-in + recovery mode + safe recommendations.	Reported low-capacity states reduce workload and offer support.
5. AI	Constrained generation + explanation + content recommendations.	AI output passes validation and safety rules.
6. ML	Data analysis + personalization experiment.	A model is evaluated against a baseline, not just added for appearance.
7. Production	Testing, monitoring, deployment, documentation.	Demo deployment is stable and reproducible.

20. MVP Definition

Do not build the whole vision for your first demo. The MVP should prove the core loop.

MVP = Account -> Assessment -> Status -> Check-in -> Next Best Action -> Quest -> Result -> XP/Progress -> Feedback

Include in MVP	Defer until core loop works
5 domains + goals	Wearables
Assessment + scoring	Voice assistant
Status dashboard	Social/community
Quests + XP	Computer vision workouts
Bottleneck engine v1	Advanced deep learning
Recovery Mode v1	Large external content catalogs
AI explanation	Complex recommendation marketplace
Feedback loop	Full mobile apps

21. Complete Example User Journey

User: student preparing for placements. **Goal:** become job-ready for a software/data role. **Current state:** low energy, high stress, inconsistent DSA practice.

System step	Decision
Assessment	Education 62, Career 48, Physical 65, Mental Wellness 44, Emotional 60.
Goal mapping	Placement goal requires DSA, SQL, projects and communication.
Bottleneck	Recovery/consistency appears to be blocking educational progress.
Check-in	Energy 3/10, stress 7/10, 45 minutes available.
Next best action	Do a 15-minute recovery activity, then one 25-minute DSA session.
Why	Recent workload and low energy make a long session less likely to succeed.
Result	User reports walk helped; completes 4 DSA problems with 8/10 quiz performance.
Adaptation	Recovery recommendation remains available; DSA quest slightly increases next session.

Notice the principle: the system optimizes the next useful step, not maximum task volume.

22. AI/ML Roadmap

AI should be introduced in layers. The project becomes stronger when you can show why each layer exists.

Stage	Technique	Purpose
1	Rules + weighted score	Transparent baseline decision engine.
2	LLM generation	Natural-language quests/explanations under constraints.
3	Analytics	Find which recommendations work best for which users/states.
4	Supervised ML	Predict quest success or recommendation acceptance using historical data.
5	Personalization	Adjust weights/difficulty per user while preserving safety constraints.
6	Experimentation	A/B test recommendation strategies against a baseline.

Avoid deep learning just to claim deep learning. An interpretable model that improves a real metric is more valuable for this project.

23. Future Expansion Map

Feature	Value	Priority
Voice System	Hands-free check-ins and guidance.	Later
Wearables	More objective activity/sleep data.	Later
Calendar integration	Use real deadlines and availability.	High
Study integrations	Import quiz/problem performance.	High
Personal knowledge base	Remember user preferences and long-term goals.	High
Mobile app	Daily usage convenience.	Later
Community	Peer accountability and cooperative challenges.	Later
Advanced content recommendation	Better recovery/learning resources.	Later

24. Major-Project Deliverables

- Working web application with deployed demo.
- Backend API with documented endpoints.
- PostgreSQL schema and migration files.
- Recommendation/decision engine documentation.
- AI integration with structured prompts and validation.
- Evaluation dataset/protocol and measured results.
- Unit, integration and safety tests.
- System architecture diagram and database ER diagram.
- Project report with problem, literature/market background, methodology, implementation, evaluation, limitations and future work.
- GitHub repository with clean README, setup instructions and screenshots.
- Short demo video and presentation.

Interview story

Explain the project as a decision and personalization system, not an “AI chatbot”. Highlight the bottleneck engine, adaptive progression, recovery mode, evidence-based feedback loop and safety architecture.

25. Definition of Done

Area	Done when...
User	Can register, complete assessment, define goals and manage privacy settings.
System state	Status page reflects current goals, domains, recent check-ins and progress.
Decision engine	Produces a deterministic candidate priority and explanation.
Quest engine	Creates a constrained, measurable action with difficulty and evidence rules.
Recovery mode	Responds appropriately to low-capacity states and avoids medical claims.
AI	Generates language inside an allowed structured schema and passes validation.
Learning loop	Stores result + feedback and changes future recommendations.
Data	Stored relationally with migrations, indexes, access rules and deletion support.
Testing	Core paths and safety scenarios are covered by automated tests.
Deployment	A reviewer can open the app and reproduce a guided demo.
Documentation	A new developer can clone, configure and run the project from the README.

26. Your Final Development Order

Follow this order to prevent the project from becoming an unstructured collection of features.

1. Learn foundations -> 2. Build backend -> 3. Add database -> 4. Build assessment -> 5. Build status -> 6. Add XP/quests -> 7. Add goals/roadmaps -> 8. Build bottleneck + next-best-action engine -> 9. Add recovery mode -> 10. Add constrained AI -> 11. Build React UI -> 12. Add analytics/ML -> 13. Test/security -> 14. Deploy -> 15. Evaluate -> 16. Document and present.

When a new feature idea appears, do not immediately code it. Add it to a "Future Features" list, score it on user value, differentiation, effort, safety and learning value, then place it into the next version only if it earns its way in.

Versioning rule

v1.0 = core loop. v1.1 = intelligence improvements. v1.2 = recovery and personalization. v2.0 = validated advanced features. This keeps the project coherent while allowing growth.

END STATE

A polished LifeSystem should feel like a supportive personal operating system: it understands context, gives one useful next step, adapts when life changes, explains its decisions, and measures progress without pretending to replace professional care.